MicroROM-NPU

RTL Module Selection and Design Rationale

Document purpose

This document records the proposed RTL module architecture for the MicroROM-NPU educational ASIC project and explains the reasoning used to select the modules. The architecture is intended to be implemented and verified incrementally before ASIC synthesis.

Project assumptions

• **Execution model:** a standalone microprogrammed execution engine for a selected subset of 32-bit RISC-V instructions; it is not a coprocessor attached to an existing CPU.

• **Control store:** a 2D vertical MicroROM organized by routine ID and local micro-step.

• **Microinstruction width:** 48 bits, as proposed in the current design.

• **ASIC implementation direction:** initially synthesize constant ROM contents into standard-cell logic rather than depend on an available memory macro.

• **Neural kernel:** a single-neuron/dense-layer computation using multiply-accumulate behavior, with multiplication implemented by a repeated-add microroutine for the first version.

1. Selected RTL modules

The following 11 hardware modules are proposed, plus one top-level integration module. This is a design plan; the modules have not all been implemented or verified yet.

No.	RTL module	Primary responsibility	Why it is needed
1	**microrom_npu_top**	Connects modules and exposes instruction/data memory, clock, reset and status interfaces.	Provides one integration boundary for simulation and ASIC synthesis.
2	**instruction_decoder**	Recognizes LB, LW, SW, ADD, ADDI, BNE, BGE and JAL; extracts fields/immediates; emits routine ID.	Separates instruction interpretation from microprogram execution.
3	**micro_rom_2d**	Returns a 48-bit microinstruction indexed by routine ID and local micro-step.	Keeps routines logically isolated and supports constant-based standard-cell synthesis.
4	**microsequencer**	Maintains routine and step state; handles sequencing, local branches, dispatch and return.	Prevents accidental fall-through between routines.
5	**microinstruction_decoder**	Decodes the 48-bit word into control signals and checks reserved encodings.	Separates field encoding from datapath behavior.
6	**architectural_register_file**	Implements x0–x31 as 32-bit registers; x0 reads zero and ignores writes.	Provides RISC-V-visible integer register state.
7	**internal_register_bank**	Holds A, B, TMP, ACC, PROD, CTR, MDR, MAR and SIGN.	Stores working state for arithmetic, memory and repeated-add multiplication.
8	**alu**	Performs selected arithmetic, comparison, negation and related operations.	Centralizes datapath operations and condition generation.
9	**program_counter**	Maintains architectural PC and supports sequential advance and branch/jump targets.	Separates architectural PC from microsequencer state.
10	**memory_interface**	Coordinates instruction/data-memory requests, reads, writes, wait states and completion.	Defines the execution engine's memory boundary.
11	**writeback_unit**	Selects/routes results to architectural registers or internal destinations.	Makes destination selection and write enables explicit.

2. How the decision was taken

The module selection was derived from the project's stated architectural goals and decomposed by responsibility. This is an engineering decomposition, not a claim that these are the only possible modules or the optimal final partition.

Design requirement	Decision taken	Rationale
Execute selected RISC-V instructions	Separate decoder and architectural register file	Instruction recognition and architectural state have different responsibilities and can be tested independently.
Control execution using microcode	Separate 2D MicroROM, microinstruction decoder and microsequencer	Stored control words, their interpretation and next-step decisions are distinct functions.
Avoid accidental transitions between routines	Index ROM by routine ID and local micro-step	Local step increments stay within the selected row; branches and routine transitions are explicit.
Use 48-bit vertical microinstructions	Dedicated microinstruction decoder	Field extraction and reserved-encoding checks remain centralized as the format evolves.
Support architectural state and control flow	Separate PC and architectural register file	Architectural PC progression differs from routine/step state.
Support loads/stores and variable memory response	Memory interface with explicit request/completion	Prevents the microprogram from assuming data is available before a read completes.
Implement arithmetic and repeated-add multiplication	ALU plus internal register bank	ALU operations are reusable; A/B/ACC/PROD/CTR hold iterative algorithm state.
Route results to the correct destination	Separate write-back control	Makes destination choice and write enables explicit and testable.
Target ASIC standard-cell synthesis	Fixed constants for initial ROM implementation	Avoids reliance on FPGA block-RAM inference or an unavailable memory macro.

3. Supporting files and verification assets

• **microrom_pkg.sv** — shared operation codes, selector encodings, field widths and constants; a package, not a hardware module.

• **Testbenches** — tests for instruction decoding, ROM lookup, microsequencer transitions, datapath operations and top-level execution.

• **Assertions/checks** — check reserved encodings, invalid-instruction dispatch prevention, routine-step bounds, x0 behavior and memory completion.

4. Recommended implementation order

1. Freeze the shared microinstruction package and field encodings.

2. Verify the decoder with directed tests, including positive and negative immediates.

3. Define routine-local step maps and termination/return conventions.

4. Implement and verify the constant 2D MicroROM.

5. Implement the microsequencer; test dispatch, local branches and returns.

6. Implement register banks, ALU, PC, memory interface and write-back.

7. Integrate the top-level, verify the instruction subset and neural arithmetic kernel, then synthesize.

5. Scope and assumptions

This is a proposed first-version architecture, not a completed or verified processor. The selected instruction subset is not full RV32I compliance. Repeated-add multiplication is an educational implementation choice, not a high-performance multiplier. A standard-cell ROM may map into gates and muxes; area and timing must be measured using the selected library and constraints. The memory protocol, reset strategy, exact microinstruction semantics and routine maps must be finalized before integration.

Document status: Architecture proposal — pending detailed interface specification, implementation and verification.